

Продолжаем изучать Dependency Injection

Получение зависимостей

В ASP.NET Core мы можем получить добавленные в приложения сервисы различными способами;

- Через свойство `Services` объекта `WebApplication` (service locator)
- Через свойство `RequestServices` контекста запроса `HttpContext` в компонентах `middleware` (service locator)
- Через конструктор класса
- Через параметр метода `Invoke` компонента `middleware`
- Через свойство `Services` объекта `WebApplicationBuilder`

Для работы определим интерфейс `ITimeService` и класс `ShortTimeService`, который реализует данный интерфейс:

```
interface ITimeService
{
    string GetTime();
}

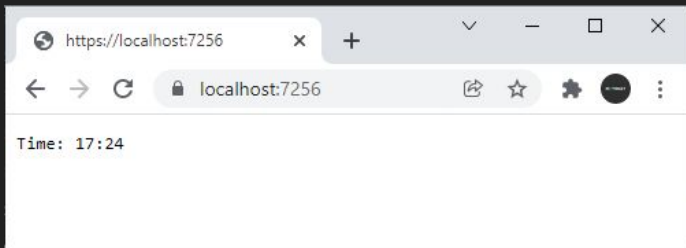
class ShortTimeService : ITimeService
{
    public string GetTime() =>
        DateTime.Now.ToShortTimeString();
}
```

Получение зависимостей. Свойство Services объекта WebApplication

Там, где нам доступен объект **WebApplication**, который представляет текущее приложение, (например, в файле Program.cs), для получения сервисов мы можем использовать его свойство **Services**. Это свойство предоставляет объект **IServiceProvider**, который предоставляет ряд методов для получения сервисов:

- `GetService<service>()`: использует провайдер сервисов для создания объекта, который представляет тип `service`. В случае если в провайдере сервисов для данного сервиса не установлена зависимость, то возвращает значение `null`
- `GetRequiredService<service>()`: использует провайдер сервисов для создания объекта, который представляет тип `service`. В случае если в провайдере сервисов для данного сервиса не установлена зависимость, то генерирует исключение

Данный паттерн получения сервиса еще называется **service locator**, и, как правило, рекомендуется использовать другие способы, но тем не менее в рамках ASP.NET Core в принципе мы можем использовать подобную функциональность особенно там, где другие способы получения зависимостей не доступны.



```
var builder = WebApplication.CreateBuilder();
```

```
builder.Services.AddTransient<ITimeService,  
ShortTimeService>();
```

```
var app = builder.Build();
```

```
app.Run(async context =>  
{  
    var timeService =  
app.Services.GetService<ITimeService>();  
    await context.Response.WriteAsync($"Time:  
{timeService?.GetTime()}");  
});
```

```
app.Run();
```

```
interface ITimeService  
{  
    string GetTime();  
}  
class ShortTimeService : ITimeService  
{  
    public string GetTime() =>  
DateTime.Now.ToShortTimeString();  
}
```

Получение зависимостей. Свойство Services объекта WebApplication

Возможна ситуация, когда сервис не будет добавлен в коллекцию сервисов, однако в какой-то части приложения мы можем попытаться его получить:

```
var builder = WebApplication.CreateBuilder();

//builder.Services.AddTransient<ITimeService, ShortTimeService>();

var app = builder.Build();

app.Run(async context =>
{
    var timeService = app.Services.GetService<ITimeService>();
    await context.Response.WriteAsync($"Time:
{timeService?.GetTime()}");
});

app.Run();
```

В этом случае переменная `timeService` будет иметь значение `null`.

Аналогичным образом можно использовать метод **GetRequiredService()** за тем исключением, что если сервис не добавлен, то метод генерирует исключение:

```
var builder = WebApplication.CreateBuilder();

builder.Services.AddTransient<ITimeService,
ShortTimeService>();

var app = builder.Build();

app.Run(async context =>
{
    var timeService =
app.Services.GetRequiredService<ITimeService>();
    await context.Response.WriteAsync($"Time:
{timeService.GetTime()}");
});

app.Run();
```

HttpContext.RequestServices

Там, где нам доступен объект **HttpContext**, мы можем использовать для получения сервисов его свойство **RequestServices**. Это свойство предоставляет объект **IServiceProvider**. То есть по сути мы имеем дело с выше описанным способом получения сервисов с помощью методов **GetService()** и **GetRequiredService()**:

```
var builder = WebApplication.CreateBuilder();

builder.Services.AddTransient<ITimeService, ShortTimeService>();

var app = builder.Build();

app.Run(async context =>
{
    var timeService = context.RequestServices.GetService<ITimeService>();
    await context.Response.WriteAsync($"Time: {timeService?.GetTime()}");
});

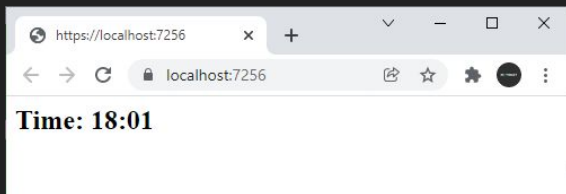
app.Run();
```

Конструкторы

Встроенная в ASP.NET Core система внедрения зависимостей использует конструкторы классов для передачи всех зависимостей. Передача сервисов через конструкторы является предпочтительным способом внедрения зависимостей.

Например, пусть в проекте определен следующий класс **TimeMessage**:

```
class TimeMessage
{
    ITimeService timeService;
    public TimeMessage(ITimeService timeService)
    {
        this.timeService = timeService;
    }
    public string GetTime() => $"Time: {timeService.GetTime()}";
}
```



Для использования класса `TimeMessage` определим следующее приложение:

```
var builder = WebApplication.CreateBuilder();

builder.Services.AddTransient<ITimeService, ShortTimeService>();
builder.Services.AddTransient<TimeMessage>();

var app = builder.Build();

app.Run(async context =>
{
    var timeMessage = context.RequestServices.GetService<TimeMessage>();
    context.Response.ContentType = "text/html; charset=utf-8";
    await context.Response.WriteAsync($"<h2>{timeMessage?.GetTime()}</h2>");
});

app.Run();

class TimeMessage
{
    ITimeService timeService;
    public TimeMessage(ITimeService timeService)
    {
        this.timeService = timeService;
    }
    public string GetTime() => $"Time: {timeService.GetTime()}";
}

interface ITimeService
{
    string GetTime();
}

class ShortTimeService : ITimeService
{
    public string GetTime() => DateTime.Now.ToShortTimeString();
}
```

Конструкторы

Для использования в приложении в качестве сервиса класс `TimeMessage` также добавляется в коллекцию сервисов. Поскольку это самодостаточная зависимость, которая представляет конкретный класс, то метод `builder.Services.AddTransient` типизируется одним этим типом `TimeMessage`. То есть классы, которые используют сервисы, сами могут выступать в качестве сервисов.

Но так как класс `TimeMessage` использует зависимость `ITimeService`, которая передается через конструктор, то нам надо также установить и эту зависимость:

```
builder.Services.AddTransient<ITimeService, ShortTimeService>();
```

И когда при обработке запроса будет использоваться класс `TimeMessage`, для создания объекта этого класса будет вызываться провайдер сервисов. Провайдер сервисов проверят конструктор класса `TimeMessage` на наличие зависимостей. Затем создает объекты для всех используемых зависимостей и передает их в конструктор.

Метод Invoke/InvokeAsync компонентов middleware

Подобно тому, как зависимости передаются в конструктор классов, точно также их можно передавать в метод **Invoke/InvokeAsync()** компонента middleware. Например, определим следующий компонент:

```
class TimeMessageMiddleware
{
    private readonly RequestDelegate next;

    public TimeMessageMiddleware(RequestDelegate next)
    {
        this.next = next;
    }

    public async Task InvokeAsync(HttpContext context, ITimeService timeService)
    {
        context.Response.ContentType = "text/html;charset=utf-8";
        await context.Response.WriteAsync($"<h1>Time: {timeService.GetTime()}</h1>");
    }
}
```

Применим компонент для обработки запроса:

```
var builder = WebApplication.CreateBuilder();

builder.Services.AddTransient<ITimeService, ShortTimeService>();

var app = builder.Build();

app.UseMiddleware<TimeMessageMiddleware>();

app.Run();

class TimeMessageMiddleware
{
    private readonly RequestDelegate next;

    public TimeMessageMiddleware(RequestDelegate next)
    {
        this.next = next;
    }

    public async Task InvokeAsync(HttpContext context, ITimeService timeService)
    {
        context.Response.ContentType = "text/html;charset=utf-8";
        await context.Response.WriteAsync($"<h1>Time: {timeService.GetTime()}</h1>");
    }
}

interface ITimeService
{
    string GetTime();
}

class ShortTimeService : ITimeService
{
    public string GetTime() => DateTime.Now.ToShortTimeString();
}
```


Метод Invoke/InvokeAsync компонентов middleware

Стоит отметить, что мы также могли бы передать зависимость и через конструктор класса middleware:

```
class TimeMessageMiddleware
{
    RequestDelegate next;
    ITimeService timeService;
    public TimeMessageMiddleware(RequestDelegate next, ITimeService timeService)
    {
        this.next = next;
        this.timeService = timeService;
    }

    public async Task InvokeAsync(HttpContext context)
    {
        context.Response.ContentType = "text/html;charset=utf-8";
        await context.Response.WriteAsync($"<h1>Time: {timeService.GetTime()}</h1>");
    }
}
```

Благодаря подобной передаче зависимости мы можем уйти от использования паттерна "service locator"

Жизненный цикл зависимостей

ASP.NET Core позволяет управлять жизненным циклом внедряемых в приложении сервисов. С точки зрения жизненного цикла сервисы могут представлять один из следующих типов:

- **Transient:** при каждом обращении к сервису создается новый объект сервиса. В течение одного запроса может быть несколько обращений к сервису, соответственно при каждом обращении будет создаваться новый объект. Подобная модель жизненного цикла наиболее подходит для легковесных сервисов, которые не хранят данных о состоянии
- **Scoped:** для каждого запроса создается свой объект сервиса. То есть если в течение одного запроса есть несколько обращений к одному сервису, то при всех этих обращениях будет использоваться один и тот же объект сервиса.
- **Singleton:** объект сервиса создается при первом обращении к нему, все последующие запросы используют один и тот же ранее созданный объект сервиса

Для создания каждого типа сервиса предназначен соответствующий метод **AddTransient()**, **AddScoped()** и **AddSingleton()**.

Для рассмотрения механизма внедрения зависимостей и жизненного цикла возьмем следующий интерфейс **ICounter**:

```
public interface ICounter
{
    int Value { get; }
}
```

Также определим реализацию этого интерфейса - класс **RandomCounter**

```
public class RandomCounter : ICounter
{
    static Random rnd = new Random();
    private int _value;
    public RandomCounter()
    {
        _value = rnd.Next(0, 1000000);
    }
    public int Value
    {
        get => _value;
    }
}
```

Жизненный цикл зависимостей

Суть класса RandomCounter состоит в генерации некоторого случайного числа в диапазоне от 0 до 1000000.

И также определим новый класс, который нам более детально поможет разобраться в механизме Dependency Injection. Этот класс назовем CounterService и определим в нем следующий код:

```
public class CounterService
{
    public ICounter Counter { get; }
    public CounterService(ICounter counter)
    {
        Counter = counter;
    }
}
```

Данный класс просто устанавливает объект ICounter, передаваемый через конструктор.

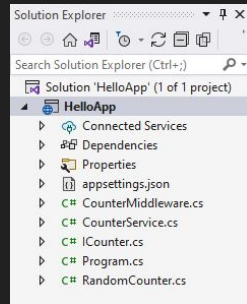
Для работы с сервисами определим компонент middleware, который назовем

CounterMiddleware

```
public class CounterMiddleware
{
    RequestDelegate next;
    int i = 0; // счетчик запросов
    public CounterMiddleware(RequestDelegate next)
    {
        this.next = next;
    }

    public async Task InvokeAsync(HttpContext httpContext,
        ICounter counter, CounterService counterService)
    {
        i++;
        httpContext.Response.ContentType =
            "text/html;charset=utf-8";
        await httpContext.Response.WriteAsync($"Запрос {i};
        Counter: {counter.Value}; Service:
        {counterService.Counter.Value}");
    }
}
```

Для получения зависимостей здесь используется метод InvokeAsync, в котором передаются две зависимости ICounter и CounterService. В самом методе выводятся значения Value из обеих зависимостей. Причем сервис CounterService сам использует зависимость ICounter.



AddTransient

Метод `AddTransient()` создает transient-объекты. Такие объекты создаются при каждом обращении к ним. Данный метод имеет ряд перегруженных версий:

- `AddTransient(Type serviceType)`
- `AddTransient(Type serviceType, Type implementationType)`
- `AddTransient(Type serviceType, Func<IServiceProvider, object> implementationFactory)`
- `AddTransient<TService>()`
- `AddTransient<TService, TImplementation>()`
- `AddTransient<TService>(Func<IServiceProvider, TService> implementationFactory)`
- `AddTransient<TService, TImplementation>(Func<IServiceProvider, TImplementation> implementationFactory)`

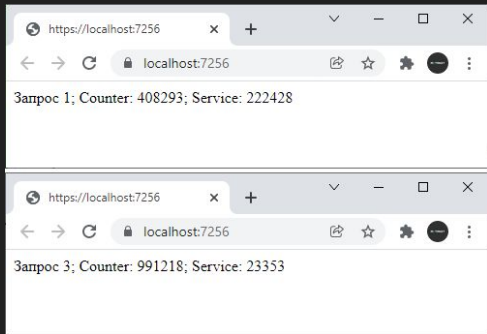
```
var builder = WebApplication.CreateBuilder();

builder.Services.AddTransient<ICounter,
RandomCounter>();
builder.Services.AddTransient<CounterService>();

var app = builder.Build();

app.UseMiddleware<CounterMiddleware>();

app.Run();
```



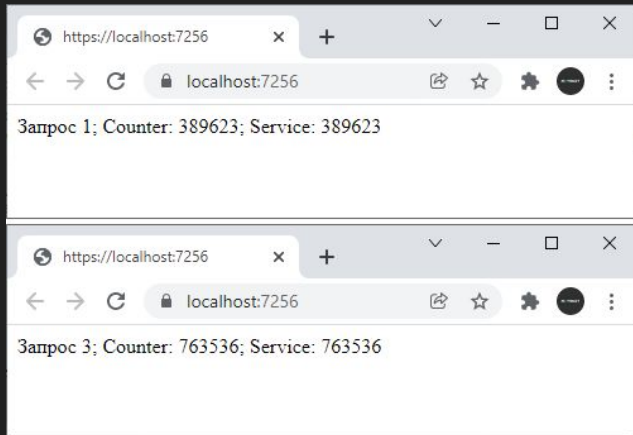
В нашем случае `CounterMiddleware` получает объект `ICounter`, для которого создается один экземпляр класса `RandomCounter`. `CounterMiddleware` также получает объект `CounterService`, который также использует `ICounter`. И для этого `ICounter` будет создаваться второй экземпляр класса `RandomCounter`. Поэтому генерируемые случайные числа обоими экземплярами не совпадают. Таким образом, применение `AddTransient` создаст два разных объекта `RandomCounter`.

При втором и последующих запросах к контроллеру будут создаваться новые объекты `RandomCounter`.

AddScoped

Метод AddScoped создает один экземпляр объекта для всего запроса. Он имеет те же перегруженные версии, что и AddTransient. Для его применения изменим код приложения следующим образом:

```
var builder =  
    WebApplication.CreateBuilder();  
  
builder.Services.AddScoped<ICounter,  
    RandomCounter>();  
builder.Services.AddScoped<CounterService  
>();  
  
var app = builder.Build();  
  
app.UseMiddleware<CounterMiddleware>();  
  
app.Run();
```



Теперь в рамках одного и того же запроса и CounterMiddleware и сервис CounterService будут использовать один и тот же объект RandomCounter. При следующем запросе к приложению будет генерироваться новый объект RandomCounter.

AddSingleton

AddSingleton создает один объект для всех последующих запросов, при этом объект создается только тогда, когда он непосредственно необходим. Этот метод имеет все те же перегруженные версии, что и AddTransient и AddScoped.

Для применения AddSingleton изменим код приложения:

```
var builder = WebApplication.CreateBuilder();

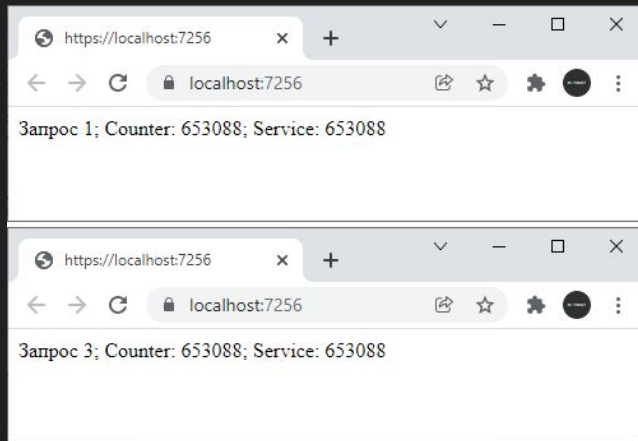
builder.Services.AddSingleton<ICounter, RandomCounter>();
builder.Services.AddSingleton<CounterService>();

var app = builder.Build();

app.UseMiddleware<CounterMiddleware>();

app.Run();
```

Как можно заметить, все три метода внедрения зависимостей имеют один и тот же принцип использования, различается только название метода. И в этом случае при нескольких последовательных запросах мы получим следующий результат



Для создания singleton-объектов необязательно полагаться на механизм Dependency Injection. Мы его можем сами создать и передать в нужный метод:

```
var builder = WebApplication.CreateBuilder();

RandomCounter rndCounter = new RandomCounter();
builder.Services.AddSingleton<ICounter>(rndCounter);
builder.Services.AddSingleton<CounterService>(new
CounterService(rndCounter));

var app = builder.Build();

app.UseMiddleware<CounterMiddleware>();

app.Run();
```

Применение сервисов в классах middleware

После добавления сервисов в коллекцию Services объекта WebApplicationBuilder они становятся доступны приложению, в том числе и в кастомных компонентах middleware. В middleware мы можем получить зависимости тремя способами:

- Через конструктор
- Через параметр метода Invoke/InvokeAsync
- Через свойство HttpContext.RequestServices

При этом надо учитывать, что компоненты middleware создаются при запуске приложения и живут в течение всего жизненного цикла приложения. То есть при последующих запросах инфраструктура ASP.NET Core использует ранее созданный компонент. И это налагает ограничения на использование зависимостей в middleware.

В частности, если конструктор передается transient-сервис, который создается при каждом обращении к нему, то при последующих запросах мы будем использовать тот же самый сервис, так как конструктор middleware вызывается один раз - при создании приложения.

```
public class TimeService
{
    public TimeService()
    {
        Time = DateTime.Now.ToLongTimeString();
    }
    public string Time { get; }
}
```

В конструкторе устанавливается свойство, которое хранит текущее время в виде строки.

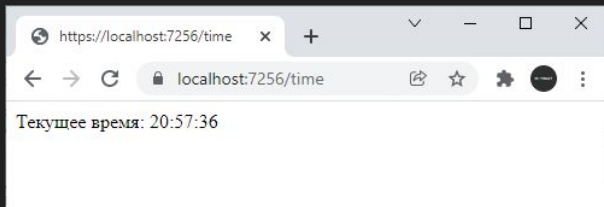
Добавим новый компонент **TimerMiddleware**, который будет использовать этот сервис для вывода времени на веб-страницу

Применение сервисов в классах middleware

```
public class TimerMiddleware
{
    RequestDelegate next;
    TimeService timeService;

    public TimerMiddleware(RequestDelegate next, TimeService timeService)
    {
        this.next = next;
        this.timeService = timeService;
    }

    public async Task InvokeAsync(HttpContext context)
    {
        if (context.Request.Path == "/time")
        {
            context.Response.ContentType = "text/html; charset=utf-8";
            await context.Response.WriteAsync($"Текущее время: {timeService?.Time}");
        }
        else
        {
            await next.Invoke(context);
        }
    }
}
```



Если сервис `TimeService` добавляется в коллекцию сервисов приложения, то мы сможем получить его через конструктор класса `TimerMiddleware`.

Логика компонента предполагает, что, если запрос пришел по адресу `/time`, то с помощью `TimeService` возвращается текущее время. Иначе мы просто обращаемся к следующему middleware в конвейере обработки запроса.

Используем этот компонент в файле `Program.cs`:

```
var builder = WebApplication.CreateBuilder();

builder.Services.AddTransient<TimeService>();

var app = builder.Build();

app.UseMiddleware<TimerMiddleware>();
app.Run(async (context) => await
context.Response.WriteAsync("Hello World"));

app.Run();
```


Применение сервисов в классах middleware

Однако сколько бы мы раз не обращались по этому пути, мы все время будем получать одно и то же время, так как объект `TimerMiddleware` был создан еще при запуске приложения и соответственно объект `TimeService` также был создан при запуске приложения. Поэтому передача через конструктор middleware больше подходит для сервисов с жизненным циклом `Singleton`, которые создаются один раз для всех последующих запросов.

Если же в middleware необходимо использовать сервисы с жизненным циклом `Scoped` или `Transient`, то лучше их передавать через параметр метода `Invoke/InvokeAsync`

```
public class TimerMiddleware
{
    RequestDelegate next;

    public TimerMiddleware(RequestDelegate next)
    {
        this.next = next;
    }

    public async Task InvokeAsync(HttpContext context, TimeService timeService)
    {
        if (context.Request.Path == "/time")
        {
            context.Response.ContentType = "text/html; charset=utf-8";
            await context.Response.WriteAsync($"Текущее время: {timeService?.Time}");
        }
        else
        {
            await next.Invoke(context);
        }
    }
}
```

Scoped-сервисы в singleton-объектах

Все объекты, которые используются в ASP.NET Core, имеют три варианта жизненного цикла. Singleton-объекты создаются один раз при первом к ним обращении, а при всех последующих запросах к приложению используется ранее созданный singleton-объект. К подобным singleton-объектам относятся, к примеру, компоненты middleware или сервисы, которые регистрируются с помощью метода `AddSingleton()`.

Transient-объекты создаются каждый раз, когда нам требуется экземпляр определенного класса. А scoped-объекты создаются по одному на каждый запрос.

Одни объекты или сервисы с помощью встроенного механизма `dependency injection` можно передать в другие объекты. Наиболее распространенный способ внедрения объектов представляется инъекция через конструктор. Однако начиная с версии ASP.NET Core 2.0 мы не можем передавать scoped-сервисы в конструктор singleton-объектов.

Например, пусть будут определены следующие классы

```
public interface ITimer
{
    string Time { get; }
}

public class Timer : ITimer
{
    public Timer()
    {
        Time = DateTime.Now.ToLongTimeString();
    }

    public string Time { get; }
}

public class TimeService
{
    private ITimer timer;

    public TimeService(ITimer timer)
    {
        this.timer = timer;
    }

    public string GetTime() => timer.Time;
}
```

Scoped-сервисы в singleton-объектах

TimeService получает через конструктор сервис ITimer и использует его для получения текущего времени.

Также пусть будет определен компонент middleware TimerMiddleware:

```
public class TimerMiddleware
{
    TimeService timeService;

    public TimerMiddleware(RequestDelegate next, TimeService timeService)
    {
        this.timeService = timeService;
    }

    public async Task Invoke(HttpContext context)
    {
        await context.Response.WriteAsync($"Time: {timeService?.GetTime()}");
    }
}
```

Компонент TimerMiddleware получает сервис TimeService и отправляет в ответ клиенту информацию о текущем времени.

TimerMiddleware является singleton-объектом. И теперь регистрируем сервис TimeService как scoped-объект:

```
var builder = WebApplication.CreateBuilder();

builder.Services.AddTransient<ITimer, Timer>();
builder.Services.AddScoped<TimeService>();

var app = builder.Build();

app.UseMiddleware<TimerMiddleware>();

app.Run();
```

Scoped-сервисы в singleton-объектах

Если мы запустим приложение, то консоль приложения нам отобразит ошибку типа "InvalidOperationException: Cannot resolve scoped service 'TimeService' from root provider."

То есть на момент создания объекта TimerMiddleware scoped-сервис TimeService еще не установлен, соответственно он использоваться не может. А без создания объекта TimeService нельзя создать объект TimerMiddleware.

Аналогичная ситуация может возникнуть, если TimeService добавляется как Transient, а сервис ITimer определен как Scoped:

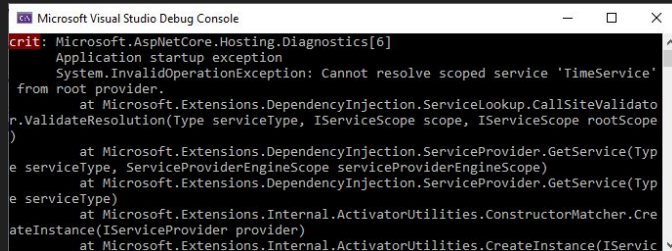
```
var builder = WebApplication.CreateBuilder();

builder.Services.AddScoped<ITimer, Timer>();
builder.Services.AddTransient<TimeService>();

var app = builder.Build();

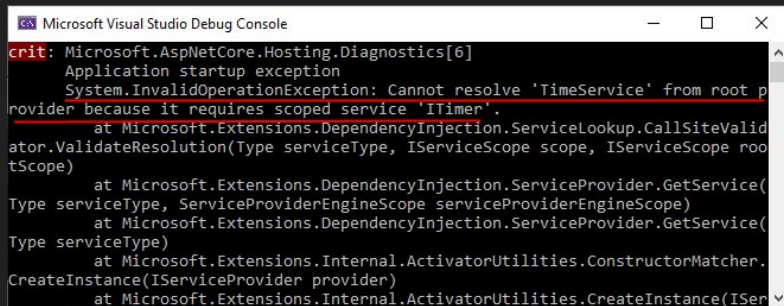
app.UseMiddleware<TimerMiddleware>();

app.Run();
```



```
Microsoft Visual Studio Debug Console
crit: Microsoft.AspNetCore.Hosting.Diagnostics[6]
Application startup exception
System.InvalidOperationException: Cannot resolve scoped service 'TimeService'
from root provider.
    at Microsoft.Extensions.DependencyInjection.ServiceLookup.CallSiteValidato
r.ValidateResolution(Type serviceType, IServiceScope scope, IServiceScope rootScope
)
    at Microsoft.Extensions.DependencyInjection.ServiceProvider.GetService(Type
serviceType, ServiceProviderEngineScope serviceProviderEngineScope)
    at Microsoft.Extensions.DependencyInjection.ServiceProvider.GetService(Type
serviceType)
    at Microsoft.Extensions.Internal.ActivatorUtilities.ConstructorMatcher.Cre
ateInstance(IServiceProvider provider)
    at Microsoft.Extensions.Internal.ActivatorUtilities.CreateInstance(IService
```

В этом случае для создания объекта TimeService надо получить сервис ITimer, но на момент вызова конструктора TimerMiddleware сервис ITimer еще неопределен



```
Microsoft Visual Studio Debug Console
crit: Microsoft.AspNetCore.Hosting.Diagnostics[6]
Application startup exception
System.InvalidOperationException: Cannot resolve 'TimeService' from root p
rovider because it requires scoped service 'ITimer'.
    at Microsoft.Extensions.DependencyInjection.ServiceLookup.CallSiteValid
ator.ValidateResolution(Type serviceType, IServiceScope scope, IServiceScope rootScope)
    at Microsoft.Extensions.DependencyInjection.ServiceProvider.GetService(
Type serviceType, ServiceProviderEngineScope serviceProviderEngineScope)
    at Microsoft.Extensions.DependencyInjection.ServiceProvider.GetService(
Type serviceType)
    at Microsoft.Extensions.Internal.ActivatorUtilities.ConstructorMatcher.
CreateInstance(IServiceProvider provider)
    at Microsoft.Extensions.Internal.ActivatorUtilities.CreateInstance(IService
```

Scoped-сервисы в singleton-объектах

Для выхода из этой ситуации ни `TimeService`, ни `ITimer` не должны иметь жизненный цикл `Scoped`. То есть это может быть `Transient` или `Singleton`.

Рассмотрим еще одну ситуацию, с которой можно столкнуться в любой части приложения, а не только в конструкторе `middleware`, когда сервис `TimeService` представляет `singleton`, а `ITimer` - `scoped`-объект:

```
var builder = WebApplication.CreateBuilder();

builder.Services.AddScoped<ITimer, Timer>();
builder.Services.AddSingleton<TimeService>();

var app = builder.Build();

app.UseMiddleware<TimerMiddleware>();

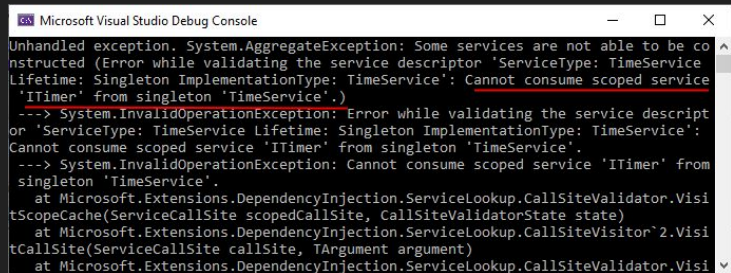
app.Run();
```

И, допустим, эти сервисы используются в `TimerMiddleware` непосредственно при обработке запроса в методе `Invoke/InvokeAsync`:

```
public class TimerMiddleware
{
    public TimerMiddleware(RequestDelegate next) { }

    public async Task Invoke(HttpContext context, TimeService timeService)
    {
        await context.Response.WriteAsync($"Time: {timeService?.GetTime()}");
    }
}
```

При запуске приложения мы опять же столкнемся с ошибкой, только немного другой "Cannot consume scoped service 'DIApp.ITimer' from singleton 'DIApp.TimeService'"



```
Microsoft Visual Studio Debug Console

Unhandled exception. System.AggregateException: Some services are not able to be constructed (Error while validating the service descriptor 'ServiceType: TimeService Lifetime: Singleton ImplementationType: TimeService': Cannot consume scoped service 'ITimer' from singleton 'TimeService'.)
---> System.InvalidOperationException: Error while validating the service descriptor or 'ServiceType: TimeService Lifetime: Singleton ImplementationType: TimeService': Cannot consume scoped service 'ITimer' from singleton 'TimeService'.
---> System.InvalidOperationException: Cannot consume scoped service 'ITimer' from singleton 'TimeService'.
    at Microsoft.Extensions.DependencyInjection.ServiceLookup.CallSiteValidator.VisitScopeCache(ServiceCallSite scopedCallSite, CallSiteValidatorState state)
    at Microsoft.Extensions.DependencyInjection.ServiceLookup.CallSiteVisitor`2.VisitCallSite(ServiceCallSite callSite, TArgument argument)
    at Microsoft.Extensions.DependencyInjection.ServiceLookup.CallSiteValidator.Visit
```

Множественная регистрация сервисов

По умолчанию при внедрении зависимостей в ASP.NET Core одна зависимость сопоставляется с одним типом. Однако бывают ситуации, когда требуется отойти от этой привязки один к одному. Первая ситуация: для одной зависимости необходимо зарегистрировать сразу несколько конкретных реализаций. Вторая ситуация: для нескольких зависимостей необходимо зарегистрировать один и тот же объект.

Регистрация для одной зависимости нескольких типов

ASP.NET Core позволяет зарегистрировать для одной зависимости сразу несколько типов.

```
var builder = WebApplication.CreateBuilder();
builder.Services.AddTransient<IHelloService, RuHelloService>();
builder.Services.AddTransient<IHelloService, EnHelloService>();

var app = builder.Build();

app.UseMiddleware<HelloMiddleware>();

app.Run();

interface IHelloService
{
    string Message { get; }
}

class RuHelloService : IHelloService
{
    public string Message => "Привет мир!";
}

class EnHelloService : IHelloService
{
    public string Message => "Hello World!";
}

class HelloMiddleware
{
    readonly IEnumerable<IHelloService> helloServices;

    public HelloMiddleware(RequestDelegate _ , IEnumerable<IHelloService> helloServices)
    {
        this.helloServices = helloServices;
    }

    public async Task InvokeAsync(HttpContext context)
    {
        context.Response.ContentType = "text/html; charset=utf-8";
        string responseText = "";
        foreach (var service in helloServices)
        {
            responseText += $"<h2>{service.Message}</h2>";
        }
        await context.Response.WriteAsync(responseText);
    }
}
```

Регистрация одного объекта для нескольких зависимостей

Теперь рассмотрим другую ситуацию: использование несколькими зависимостями одного и того же объекта. Сначала рассмотрим ситуацию, с которой мы можем столкнуться. Допустим, у нас есть следующие объекты

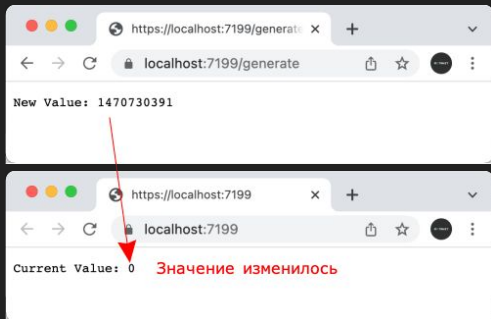
```
interface IGenerator
{
    int GenerateValue();
}

interface IReader
{
    int ReadValue();
}

class ValueStorage : IGenerator, IReader
{
    int value;
    public int GenerateValue()
    {
        value = new Random().Next();
        return value;
    }

    public int ReadValue() => value;
}
```


Регистрация одного объекта для нескольких зависимостей



```
var builder = WebApplication.CreateBuilder();  
builder.Services.AddSingleton<IGenerator, ValueStorage>();  
builder.Services.AddSingleton<IReader, ValueStorage>();
```

```
var app = builder.Build();
```

```
app.UseMiddleware<GeneratorMiddleware>();  
app.UseMiddleware<ReaderMiddleware>();
```

```
app.Run();
```

```
class GeneratorMiddleware  
{  
    RequestDelegate next;  
    IGenerator generator;  
  
    public GeneratorMiddleware(RequestDelegate next, IGenerator  
generator)  
    {  
        this.next = next;  
        this.generator = generator;  
    }  
  
    public async Task InvokeAsync(HttpContext context)  
    {  
        if (context.Request.Path == "/generate")  
            await context.Response.WriteAsync($"New Value:  
{generator.GenerateValue()}");  
        else  
            await next.Invoke(context);  
    }  
}
```

```
class ReaderMiddleware  
{  
    IReader reader;  
  
    public ReaderMiddleware(RequestDelegate _, IReader  
reader) => this.reader = reader;  
  
    public async Task InvokeAsync(HttpContext context)  
    {  
        await context.Response.WriteAsync($"Current Value:  
{reader.ReadValue()}");  
    }  
}  
  
interface IGenerator  
{  
    int GenerateValue();  
}  
  
interface IReader  
{  
    int ReadValue();  
}  
  
class ValueStorage : IGenerator, IReader  
{  
    int value;  
    public int GenerateValue()  
    {  
        value = new Random().Next();  
        return value;  
    }  
  
    public int ReadValue() => value;  
}
```

Регистрация одного объекта для нескольких зависимостей

Для исправления ситуации нам надо определить один объект для обоих зависимостей

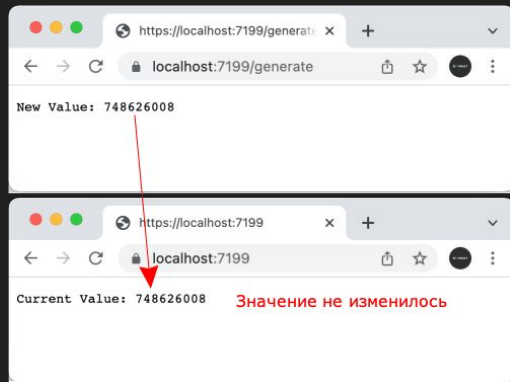
Теперь определяем один объект ValueStorage:

```
var valueStorage = new ValueStorage();
```

Затем передаем его в сервисы:

```
builder.Services.AddSingleton<IGenerator>(valueStorage);
```

```
builder.Services.AddSingleton<IReader>(valueStorage);
```



```
var builder = WebApplication.CreateBuilder();
```

```
var valueStorage = new ValueStorage();
```

```
builder.Services.AddSingleton<IGenerator>(valueStorage);
```

```
builder.Services.AddSingleton<IReader>(valueStorage);
```

```
var app = builder.Build();
```

```
app.UseMiddleware<GeneratorMiddleware>();
```

```
app.UseMiddleware<ReaderMiddleware>();
```

```
app.Run();
```

```
class GeneratorMiddleware
```

```
{
```

```
    RequestDelegate next;
```

```
    IGenerator generator;
```

```
    public GeneratorMiddleware(RequestDelegate next, IGenerator generator)
```

```
    {
```

```
        this.next = next;
```

```
        this.generator = generator;
```

```
    }
```

```
    public async Task InvokeAsync(HttpContext context)
```

```
    {
```

```
        if (context.Request.Path == "/generate")
```

```
        {
            await context.Response.WriteAsync($"New Value: {generator.GenerateValue()}");
```

```
        }
        else
```

```
        {
            await next.Invoke(context);
```

```
        }
```

```
    }
```

```
class ReaderMiddleware
```

```
{
```

```
    IReader reader;
```

```
    public ReaderMiddleware(RequestDelegate _, IReader reader) => this.reader = reader;
```

```
    public async Task InvokeAsync(HttpContext context)
```

```
    {
```

```
        await context.Response.WriteAsync($"Current Value: {reader.ReadValue()}");
```

```
    }
```

```
interface IGenerator
```

```
{
```

```
    int GenerateValue();
```

```
}
```

```
interface IReader
```

```
{
```

```
    int ReadValue();
```

```
}
```

```
class ValueStorage : IGenerator, IReader
```

```
{
```

```
    int value;
```

```
    public int GenerateValue()
```

```
    {
```

```
        value = new Random().Next();
```

```
        return value;
```

```
    }
```

```
    public int ReadValue() => value;
```

```
}
```

Регистрация одного объекта для нескольких зависимостей

В качестве альтернативы можно было бы определить `ValueStorage` как сервис-синглтон и затем при регистрации других зависимостей получить этот сервис:

```
var builder = WebApplication.CreateBuilder();

builder.Services.AddSingleton<ValueStorage>();
builder.Services.AddSingleton<IGenerator>(serv => serv.GetRequiredService<ValueStorage>());
builder.Services.AddSingleton<IReader>(serv => serv.GetRequiredService<ValueStorage>());
```

Маршрутизация. Конечные точки. Метод Map

Система маршрутизации отвечает за сопоставление входящих запросов с маршрутами и на основании результатов сопоставления выбирает для обработки запроса определенную **конечную точку** приложения. **Конечная точка** или **endpoint** представляет некоторый код, который обрабатывает запрос. По сути конечная точка объединяет шаблон маршрута, которому должен соответствовать запрос, и обработчик запроса по этому маршруту.

ASP.NET Core по умолчанию предоставляет простой и удобный функционал для создания конечных точек. Ключевым типом в этом функционале является интерфейс **Microsoft.AspNetCore.Routing.IEndpointRouteBuilder**. Он определяет ряд методов для добавления конечных точек в приложение. И поскольку класс **WebApplication** также реализует данный интерфейс, то соответственно все методы интерфейса мы можем вызывать и у объекта **WebApplication**.

Для использования системы маршрутизации в конвейер обработки запроса добавляются два встроенных компонента middleware:

- **Microsoft.AspNetCore.Routing.EndpointMiddleware** добавляет в конвейер обработки запроса конечные точки. Добавляется в конвейер с помощью метода **UseEndpoints()**
- **Microsoft.AspNetCore.Routing.EndpointRoutingMiddleware** добавляет в конвейер обработки запроса функциональность сопоставления запросов и маршрутов. Данный middleware выбирает конечную точку, которая соответствует запросу и которая затем обрабатывает запрос. Добавляется в конвейер с помощью метода **UseRouting()**

Причем обычно не требуется явным образом подключать эти два компонента middleware. Объект **WebApplicationBuilder** автоматически сконфигурирует конвейер таким образом, что эти два middleware добавляются при использовании конечных точек.

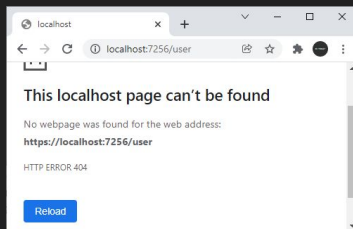
Метод Map

Самым простым способом определения конечной точки в приложении является метод **Map**, который реализован как метод расширения для типа `IEndpointRouteBuilder`. Он добавляет конечные точки для обработки запросов типа **GET**. Данный метод имеет три версии:

```
public static RouteHandlerBuilder Map (this IEndpointRouteBuilder endpoints,
RoutePattern pattern, Delegate handler);
public static IEndpointConventionBuilder Map (this IEndpointRouteBuilder endpoints,
string pattern, RequestDelegate requestDelegate);
public static RouteHandlerBuilder Map (this IEndpointRouteBuilder endpoints, string
pattern, Delegate handler);
```

В всех трех реализациях этот метод в качестве параметра `pattern` принимает шаблон маршрута, которому должен соответствовать запрос. Данный параметр может представлять тип `RoutePattern` или `string`.

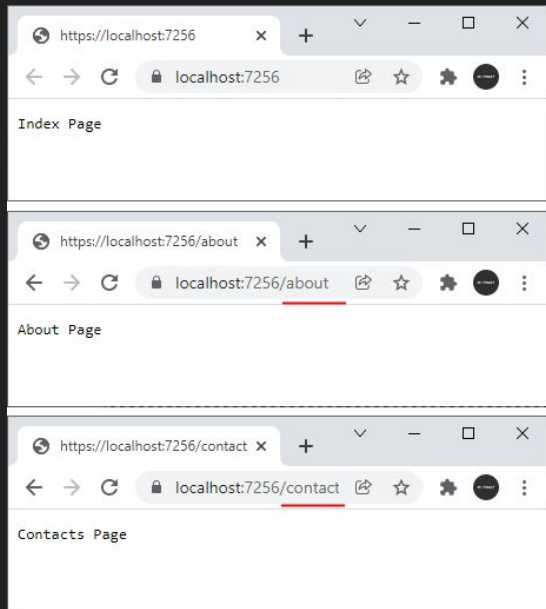
Последний параметр представляет действие, которое будет обрабатывать запрос. Это может быть делегат типа `RequestDelegate`, либо делегат `Delegate`.



```
var builder =
WebApplication.CreateBuilder();
var app = builder.Build();

app.Map("/", () => "Index Page");
app.Map("/about", () => "About Page");
app.Map("/contact", () => "Contacts Page");

app.Run();
```



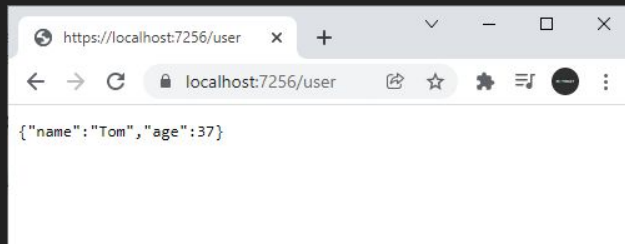
Метод Map

Ранее обработчики маршрутов возвращали строки, но в принципе это может быть любое значение, например:

```
var builder =  
WebApplication.CreateBuilder();  
var app = builder.Build();  
  
app.Map("/", () => "Index Page");  
app.Map("/user", () => new Person("Tom",  
37));  
  
app.Run();
```

```
record class Person(string Name, int Age);
```

Здесь обработчик запроса второй конечной точки возвращает в ответ объект Person. По умолчанию подобные объекты при отправке сериализуются в JSON:



В принципе можно ничего из обработчика не возвращать, просто выполнять некоторые действия:

```
var builder =  
WebApplication.CreateBuilder();  
var app = builder.Build();  
  
app.Map("/", () => "Index Page");  
app.Map("/user",  
()=>Console.WriteLine("Request Path:  
/user"));  
  
app.Run();
```

В данном случае в обработчике второй конечной точки просто логируем на консоль некоторую информацию.

Метод Map

При необходимости обработчик маршрута можно вынести в полноценный метод:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();
```

```
app.Map("/", IndexHandler);
app.Map("/user", UserHandler);
```

```
app.Run();
```

```
string IndexHandler()
{
    return "Index Page";
}

Person UserHandler()
{
    return new Person("Tom", 37);
}

record class Person(string Name, int Age);
```

В примерах выше обработчик запроса представлял делегат Delegate. Если же необходимо получить полный доступ к контексту HttpContext, то можно использовать другую версию метода, которая принимает делегат **RequestDelegate**:

```
var builder =
WebApplication.CreateBuilder();
var app = builder.Build();
```

```
app.Map("/", () => "Index Page");
app.Map("/about", async (context) =>
{
    await context.Response.WriteAsync("About
Page");
});
app.Run();
```

Получение всех маршрутов приложения

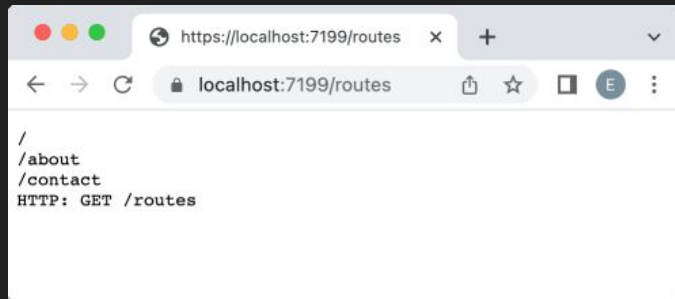
ASP.NET Core позволяет легко получить все имеющиеся в приложении конечные точки. Так, определим следующий код:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.Map("/", () => "Index Page");
app.Map("/about", () => "About Page");
app.Map("/contact", () => "Contacts Page");

app.MapGet("/routes", (IEnumerable<EndpointDataSource>
endpointSources) =>
    string.Join("\n", endpointSources.SelectMany(source =>
source.Endpoints)));

app.Run();
```



Получение всех маршрутов приложения

```
using System.Text;

var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.Map("/", () => "Index Page");
app.Map("/about", () => "About Page");
app.Map("/contact", () => "Contacts Page");

app.MapGet("/routes", (IEnumerable<EndpointDataSource> endpointSources) =>
{
    var sb = new StringBuilder();
    var endpoints = endpointSources.SelectMany(es => es.Endpoints);
    foreach (var endpoint in endpoints)
    {
        sb.AppendLine(endpoint.DisplayName);

        // получим конечную точку как RouteEndpoint
        if (endpoint is RouteEndpoint routeEndpoint)
        {
            sb.AppendLine(routeEndpoint.RoutePattern.RawText);
        }

        // получение метаданных
        // данные маршрутизации
        // var routeNameMetadata = endpoint.Metadata.Of<Type<Microsoft.AspNetCore.Routing.RouteNameMetadata>().FirstOrDefault();
        // var routeName = routeNameMetadata?.RouteName;
        // данные http - поддерживаемые типы запросов
        //var httpMethodsMetadata = endpoint.Metadata.Of<Type<HttpMethodMetadata>().FirstOrDefault();
        //var httpMethods = httpMethodsMetadata?.HttpMethods; // [GET, POST, ...]
    }
    return sb.ToString();
});

app.Run();
```

Параметры маршрута

Шаблон маршрута, который сопоставляется с конечной точкой, может иметь **параметры**. Параметры имеют имя и определяются в шаблоне маршрута внутри фигурных скобок: {название_параметра}

Например, определим конечную точку, которая принимает параметр:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.Map("/users/{id}", (string id) => $"User Id: {id}");
app.Map("/users", () => "Users Page");
app.Map("/", () => "Index Page");

app.Run();
```

Здесь определено три конечных точки, которые используются шаблоны маршрутов `"/users/{id}"`, `"/users"` и `"/"`. Причем шаблон маршрута для первой конечной точки принимает параметр с именем `id`:

```
"/users/{id}"
```

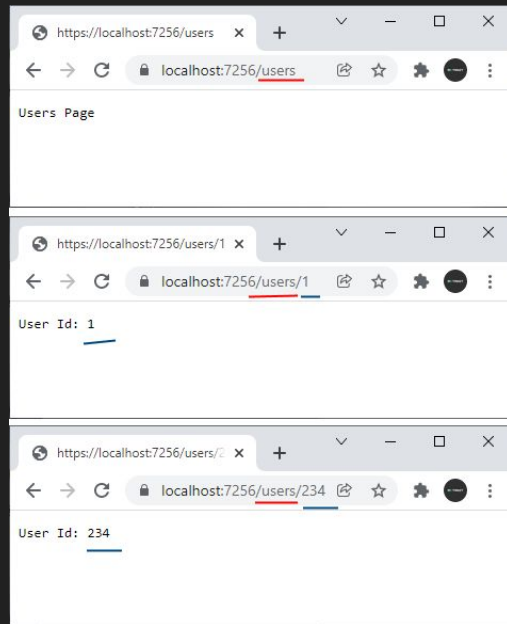
Параметр `id` будет представлять содержимое последнего сегмента. Например, в пути запроса `"/users/134"` последний сегмент `"134"` как раз будет представлять параметр `id`. Если же параметр `id` не передается, а путь запроса - `"/users"`, то такой запрос обрабатывается второй конечной точкой.

Если маршрут принимает параметр, то в делегате-обработчике маршрута мы можем получить значение этого параметра. Для этого мы можем просто определить действие, которое принимает соответствующий параметр:

```
(string id) => $"User Id: {id}"
```

То есть здесь через параметр `id` мы получаем одноименный параметр маршрута и далее можем его использовать при обработке запроса. Стоит отметить, что название параметра маршрута и название параметра лямбда-выражения совпадают.

Также стоит отметить, что здесь для параметра `id` указан тип - тип `string` (то есть здесь мы ожидаем получить через параметр маршрута некоторую строку). Поскольку метод `Map` может принимать в качестве второго параметра делегат `RequestDelegate`, который тоже принимает один параметр - `HttpContext`, то, если не указать тип, то делегат-обработчик будет рассматриваться именно как `RequestDelegate`.



Определение нескольких параметров

Подобным образом можно определять и большее количество параметров маршрута:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.Map(
    "/users/{id}/{name}",
    (string id, string name) => $"User Id: {id}    User Name: {name}"
);
app.Map("/users", () => "Users Page");
app.Map("/", () => "Index Page");

app.Run();
```

В данном случае маршрут первой конечной точки принимает два параметра - id и name

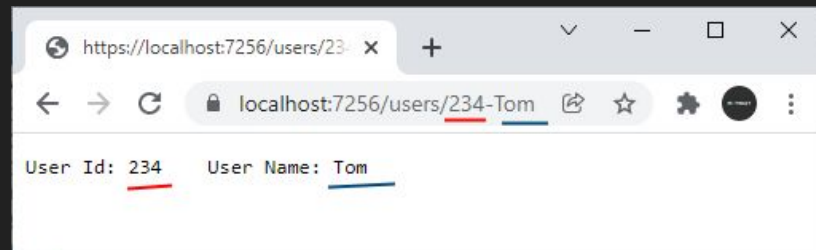
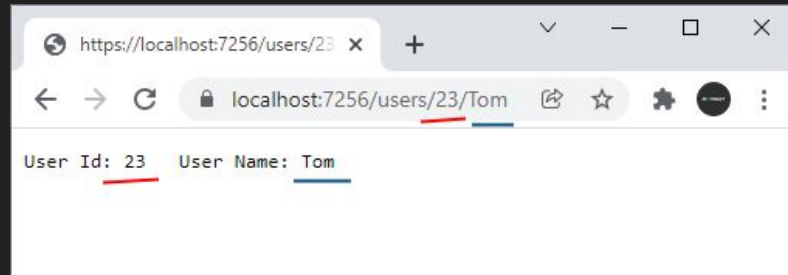
Следует учитывать, что если параметры маршрута определяются подряд, то между ними должен располагаться разделитель. Например, следующий шаблон маршрута работать не будет:

```
"/users/{id}{name}"
```

так как между параметрами id и name нет разделителя.

Обычно в качестве разделителя выступает слеш, который оформляет отдельный сегмент в пути запроса. Но это необязательно. В качестве разделителя могут применяться и другие символы. Например, установим в качестве разделителя дефис:

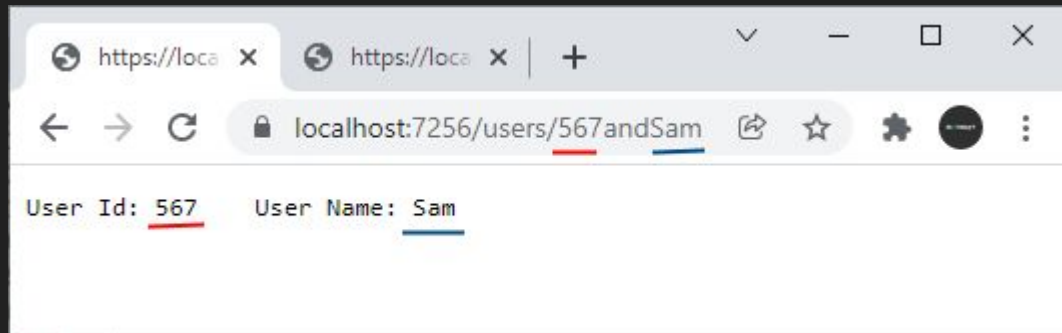
```
app.Map(
    "/users/{id}-{name}",
    (string id, string name) => $"User Id: {id}    User Name: {name}"
);
```



Определение нескольких параметров

Более того это может быть набор символов:

```
app.Map(  
    "/users/{id}and{name}",  
    (string id, string name) => $"User Id: {id}    User Name: {name}"  
);
```



Вынесение обработчика маршрута в отдельный метод

Если обработчик маршрута выносится в отдельный метод, то он также может определять параметры, который сопоставляются с параметрами маршрута по имени:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.Map("/users/{id}/{name}", HandleRequest);
app.Map("/users", () => "Users Page");
app.Map("/", () => "Index Page");

app.Run();

string HandleRequest(string id, string name)
{
    return $"User Id: {id}    User Name: {name}";
}
```

Необязательные параметры маршрута

Параметры маршрута могут быть необязательными. Чтобы определить параметр как необязательный, после его названия указывается знак вопроса. Например, определим следующее приложение:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

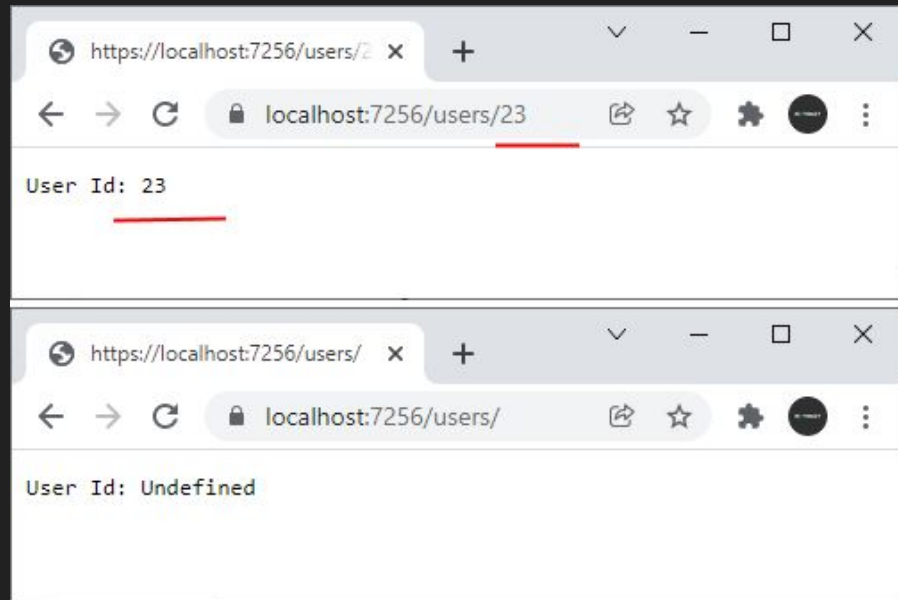
app.Map("/users/{id?}", (string? id) => $"User Id: {id??"Undefined"}");
app.Map("/", () => "Index Page");

app.Run();
```

Шаблон маршрута первой конечной точки состоит из двух сегментов, причем второй сегмент представляет параметр id, который помечен как необязательный. А это значит, что мы можем в запросе игнорировать значение для этого сегмента. Например, данный шаблон будет соответствовать двум следующим url:

```
/users/
/users/23
```

Необязательные параметры следует помещать в конце шаблона маршрута, как в случае с параметром id в примере выше. То есть, к примеру, шаблон "/users/{id?}/{name}" не будет работать корректно, если мы для параметра id не передадим значение. А вот шаблон "/users/{name}/{id?}" будет работать нормально.



Значение параметра по умолчанию

Параметрам маршрута также можно назначить значения по умолчанию на случай, если им не переданы значения:

```
var builder =  
    WebApplication.CreateBuilder();  
var app = builder.Build();  
  
app.Map(  
  
    "{controller=Home}/{action=Index}/{id?}",  
    (string controller, string action,  
    string? id) =>  
        $"Controller: {controller} \nAction:  
{action} \nId: {id}"  
);
```

```
app.Run();
```

Здесь определена одна конечная точка, которая использует следующий шаблон маршрута:

```
"{controller=Home}/{action=Index}/{id?}"
```

То есть шаблон состоит из трех параметров. Параметр "controller" имеет значение по умолчанию "Home". Параметр "action" имеет значение по умолчанию "Index". Параметр "id" определен как необязательный. В итоге при различных запросах у нас получатся следующие значения

https://localhost:7256	Controller: Home Action: Index Id:
https://localhost:7256/Book/	Controller: Book Action: Index Id:
https://localhost:7256/Book/Show/	Controller: Book Action: Show Id:
https://localhost:7256/Book/Show/2	Controller: Book Action: Show Id: 2

Запрос	Параметры запроса
https://localhost:7256/	controller=Home action=Index
https://localhost:7256/Book	controller=Book action=Index
https://localhost:7256/Book/Show	controller=Book action=Show
https://localhost:7256/Book/Show/2	controller=Book action=Show id=2

Передача произвольного количества параметров

Мы можем обозначить любое количество сегментов в запросе, чтобы не быть жестко привязанным к числу сегментов с помощью параметра со знаком * ("звездочка") или ** (две звездочки) (это так называемый catchall-параметр):

```
var builder = WebApplication.CreateBuilder();  
var app = builder.Build();
```

```
app.Map("users/{**info}", (string info) => $"User  
Info: {info}");
```

```
app.Map("/", () => "Index Page");
```

```
app.Run();
```

Для первой конечной точки определяется шаблон маршрута, в котором параметр `info` будет представлять все, что идет после `/users`. Это может быть в том числе пустая строка.

